

INSTRUCTIONS

1. Submission is only via Themis for the practical exercises and via Brightspace for the theoretical exercises. Deadlines are strict.
 2. The exercises in this assignment add up to 100 points. To calculate your grade simply divide the number of points by 10.
 3. Seeking solutions from the internet, from any external resource, or from any other person is prohibited.
 4. Please note that the course lecturer reserves the right to ask the student submitting the assignment to explain the answers to any or all questions. If the student is unable to provide a satisfactory answer then that question may receive partial/no credit.
 5. Of course, university policies on plagiarism always apply. In particular, any suspected plagiarism will be reported to the Board of Examiners.
-

Practical Exercises

A note on how Themis works

For some exercise you will be given input via registers (similar to how you did in the previous assignment), and for some exercises you will be given input via the console! The console is available to you in LC3.

For some exercises in this lab **efficiency is important** and will be taken into account! Therefore, if your program is not fast enough, you will fail with a timeout which simply means your program took too many steps to execute. If you are curious about the upper limit of steps you can check out the compile.log file. Nevertheless, all the exercises have enough of an upper bound to where you should never have to worry about optimizing the number of instructions but rather your algorithm overall.

The main focus of this assignment is implementing your own functions (subroutines). Themis will accept your submissions without subroutines, however we **strongly recommend** using subroutines, as further exercises become very very difficult to solve without them. We will not enforce a specific function style, and you are free to use anything you want.

Important Note! In LC-3 by default you do not see what you type! For simplicity sake, we will be printing every single part of the input (In practice: every GETC is followed by a PUTC/OUT). This can make the output a bit messy to read. If you have any questions feel free to contact a TA! For the sake of clarity, whenever an output contains text which comes from the input being repeated, it will be **marked with bold**.

1. Reading Input

In this problem you will be given a number via your console. The number is **always positive**, and will not exceed $2^{15} - 1$ (thus ensuring that there will never be overflow in LC3). Once all digits are types, the number will be followed either by a space or by a newline! The output is then expected to be placed in R1. In this problem you do not have to print anything to the console (although adding the PUTC after each GETC will make it easier to debug), as the output will be checked through the register. **(10 points)**

Example 1

Input: (via console)

5

Output: (via registers)

R1: 5

Example 2**Input: (via console)**

10

Output: (via registers)

R1: 10

Example 3**Input: (via console)**

123

Output: (via registers)

R1: 123

2. Divmod

In this exercise you will be implementing a function named divmod! This function calculates both the quotient and the remainder of two numbers (where the first one is divided by a second one). You will be given input through registers R0 and R1, and you are expected to place the result of the divmod operation within R2 (the quotient) and R3 (the remainder). It is guaranteed that both numbers are positive and the divisor will never be 0 (essentially only inputs which make sense are provided). **(10 points)**

Example 1**Input: (via registers)**R0: 8
R1: 5**Output: (via registers)**R2: 1
R3: 3**Example 2****Input: (via registers)**R0: 30
R1: 10**Output: (via registers)**R2: 3
R3: 0

3. Printing Numbers

In this exercise you will be given two numbers via console, and you simply need to print every number between them (inclusive on both ends). Be careful with the formatting of the output, as there should be a space between numbers, no space after the last number and a newline after all numbers have been printed! It is guaranteed that the 2nd number is at least equal to or greater than the first one and that none of the numbers are negative. (there will always be at least one number to print) **(25 points)**

Example 1**Input: (via console)**

```
1 5
```

Output: (via console)

```
1 5
1 2 3 4 5
```

Example 2**Input: (via console)**

```
1 9
```

Output: (via console)

```
1 9
1 2 3 4 5 6 7 8 9
```

4. Is Number Prime?

In this exercise you will be given a number via console, (again being followed by either a space or a newline), and you have to write a program which figures out if the number is prime or not. If the number is prime, it should print `x is a prime number`, otherwise it should print `x is not a prime number as it is divisible by y`. **(25 points)**

Example 1**Input: (via console)**

```
3
```

Output: (via console)

```
3
3 is a prime number
```

Example 2**Input: (via console)**

```
5
```

Output: (via console)

```
5
5 is a prime number
```

Example 3**Input: (via console)**

```
24
```

Output: (via console)

```
24
```

```
24 is not a prime number as it is divisible by 2
```

5. Are Numbers Prime?

We have spent quite a bit of time building a couple functions, and it is time to put them to use! In this exercise, you will be given a number n , followed by n numbers, which you have to check for being prime. The output is identical to the previous exercise, but will show the message for each of the numbers. **(10 points)**

Example 1 (addition)**Input: (via console)**

```
2
7 20
```

Output: (via console)

```
2
```

```
7 20
```

```
7 is a prime number
```

```
20 is not a prime number as it is divisible by 2
```

Example 2**Input: (via console)**

```
10
2 3 4 5 6 7 8 9 10 11
```

Output: (via console)

```
10
```

```
2 3 4 5 6 7 8 9 10 11
```

```
2 is a prime number
```

```
3 is a prime number
```

```
4 is not a prime number as it is divisible by 2
```

```
5 is a prime number
```

```
6 is not a prime number as it is divisible by 2
```

```
7 is a prime number
```

```
8 is not a prime number as it is divisible by 2
```

```
9 is not a prime number as it is divisible by 3
```

```
10 is not a prime number as it is divisible by 2
```

```
11 is a prime number
```

Example 3**Input: (via console)**

```
3
2 3 5
```

Output: (via console)

```
3
2 3 5
2 is a prime number
3 is a prime number
5 is a prime number
```

Theoretical Exercises

6. You are given the following LC3 ASM program! For this exercise, you have two tasks: **(10 points)**
1. Write the symbol table (use an absolute address, not offsets)
 2. Manually assemble (aka convert to the internal LC3 representation) the instructions at A, B and D. Both binary and hex are fine.

```
.ORIG x3000
    AND R0, R0, #0
A   LD R1, E
    AND R2, R1, #1
    BRp C
B   ADD R1, R1, #-1
C   ADD R0, R0, R1
    ADD R1, R1, #-2
D   BRp C
    ST R0, F
    TRAP x25
E   .BLKW 1
F   .BLKW 1
    .END
```

7. For this exercise we will assume that two new operations have been added to LC3: PUSH and POP. The instruction PUSH Rx pushes the value in Register x onto the stack. POP Rx removes a value from the stack and loads it into Rx. You are also given a list of the instructions which were executed, however some of the registers went missing. Fill out what the correct registers would be! **(10 points)**

BEFORE		AFTER	
R0	x0000	R0	x1111
R1	x1111	R1	x1111
R2	x2222	R2	x3333
R3	x3333	R3	x3333
R4	x4444	R4	x4444
R5	x5555	R5	x5555
R6	x6666	R6	x6666
R7	x7777	R7	x4444

Operations:

```
PUSH R4
PUSH (a)
POP (b)
PUSH (c)
POP R2
POP (d)
```